Artifact Narrative: Enhancement Three: Databases

Artifact Description

The artifact is an interactive three-dimensional desktop scene that I originally created in CS 330: Computational Graphics and Visualization. The project uses C++, OpenGL, GLFW, GLEW, GLM, textures, materials, lighting, and reusable primitive meshes to display a table, cup, candle, succulent plant, and laptop. The original CS 330 version was created as a graphics course project. During CS 499, I used it as one continuous artifact to demonstrate growth across software engineering and design, algorithms and data structures, and databases. The software engineering enhancement improved its organization, validation, resource management, documentation, lighting, and dependency configuration. The algorithms enhancement added a logical object registry, hash-based lookup, ray casting, bounding-box intersection, nearest-hit selection, and visual highlighting.

For Milestone Four, I added a SQLite persistence layer that converts the scene from an application whose logical state exists only in memory into one that can create, retrieve, update, list, and delete saved scenes. The application now creates a local scene_portfolio.db file and provides keyboard commands to save the current scene, reload it, list saved scenes, and delete a scene. The enhanced submission includes the original Milestone Three baseline, the database-enhanced project, the SQL schema, test cases, design notes, and build instructions.

Justification for Inclusion

I selected this artifact because persistence was a clear limitation in the earlier versions. The scene could render objects and identify a selected object, but its logical object records, model matrices, bounds, and selection state disappeared when the program closed. Adding a

database addresses a practical user need and shows how a graphics application can be connected to a durable data model. It also allows the three enhancements in my portfolio to build on one another: the modular design makes database integration safer, and the SceneObject data structure supplies the records that the database stores.

Database Design and Software Development Skills

The database uses a normalized relational design. The scenes table stores one record per named scene. The scene_objects table stores the objects belonging to a scene and uses a foreign key to represent the one-to-many relationship. This separation prevents scene names and timestamps from being repeated for every object. The child table stores a stable object key, display name, selection state, local bounding-box values, and all sixteen model-matrix values. A unique constraint prevents duplicate object keys within one scene, an index supports lookups by scene identifier, and ON DELETE CASCADE removes dependent objects when their parent scene is deleted. A third table provides a controlled location for future application settings.

The DatabaseManager class owns the database connection and exposes focused operations for opening the database, initializing the schema, saving, loading, listing, and deleting scenes. The scene manager delegates persistence work to this class instead of mixing SQL with rendering code. On Windows, the manager loads the operating system's winsqlite3.dll at runtime. This design avoids adding another import library or copied DLL to the Visual Studio project, which is an important improvement after resolving earlier GLEW dependency problems. On Linux, the same implementation loads libsqlite3, allowing the database behavior to be tested independently of the Windows graphics environment.

CRUD Operations Transactions and Security

The enhancement demonstrates the complete CRUD cycle. SaveScene creates a new scene or updates an existing scene and replaces its child object rows. LoadScene joins the parent and child tables and reconstructs each SceneObject. ListScenes uses grouping and an aggregate count to return useful scene summaries. DeleteScene removes a parent record and relies on foreign-key cascading to remove its objects. Save operations run inside an immediate transaction. If any parent lookup, deletion, binding, or object insertion fails, the transaction rolls back, so the database cannot contain a partially replaced scene.

Security and defensive behavior were included throughout the design. All variable text is passed through prepared statements and bound parameters rather than concatenated into SQL. This prevents quotation marks or SQL-like text in a scene name from changing the structure of a command. The schema uses NOT NULL, UNIQUE, CHECK, and FOREIGN KEY constraints as a second validation layer. Application code rejects empty or oversized names, control characters, invalid object identifiers, invalid display names, and nonfinite transformation values before beginning a transaction. SQLite is opened in full-mutex mode, a busy timeout reduces immediate lock failures, foreign-key enforcement is enabled for each connection, and database errors are reported without exposing sensitive information or terminating the rendering loop. Loaded objects must also pass the SceneObjectRegistry validation before replacing the active collection.

Course Outcome Progress

This enhancement most directly meets Course Outcome Four, which requires the use of well-founded techniques, skills, and tools to implement computing solutions that deliver value. I

used C++, SQLite, SQL constraints, relational modeling, prepared statements, transactions, aggregate queries, indexing, foreign keys, and runtime library loading to add durable scene management to an existing application. The result delivers visible value because users can preserve and restore state instead of rebuilding it every time the program starts.

The enhancement also provides substantial evidence for Course Outcome Five. I anticipated injection attempts, malformed records, partial writes, duplicate identifiers, invalid transformations, database locks, unavailable libraries, and orphaned child rows. Parameter binding, layered validation, rollback behavior, foreign-key enforcement, cascading deletion, full-mutex mode, and safe error handling mitigate those risks. The injection-style test saves a name containing quotation marks and a DROP TABLE phrase, then confirms that the schema and other records remain intact.

Course Outcome Three is supported through the data-model and query trade-offs. I selected SQLite instead of a client-server database because the artifact is a local, single-user desktop application. I used explicit matrix columns instead of a binary blob because the values remain inspectable and queryable, although a blob could be more compact. I selected transactional row replacement because it is simple and reliable for five objects; a large collaborative editor might instead need per-object versioning, upserts, and conflict resolution. The indexed foreign key improves scene retrieval without adding unnecessary indexes to every numeric column.

Course Outcome Two is demonstrated through the narrative, source comments, schema file, test matrix, console messages, and instructions adapted to technical reviewers and application users. Course Outcome One is supported by the separation of responsibilities and the documented schema and interface, which make it easier for another developer to review the

design, discuss trade-offs, and extend the persistence layer without modifying shader or mesh code. My outcome-coverage plan has not changed substantially. This milestone strengthens Outcomes Three, Four, and Five, while the final professional self-assessment and complete ePortfolio will connect the artifacts more explicitly to collaboration, stakeholder communication, and my professional goals.

Reflection on the Enhancement Process

This enhancement taught me that persistence requires coordination between the application's data structures, database schema, validation rules, transactions, and user controls. I had to determine which object fields should be stored, define the relationship between scenes and objects, and ensure that failed updates would not damage valid data. I also avoided another dependency issue by dynamically loading SQLite instead of adding a new library path to Visual Studio.

Another challenge was deciding how to store each object's transformation matrix. I selected sixteen numeric columns because they preserve data types and make individual values easy to inspect. Ten automated tests verified the schema, CRUD operations, validation, cascading deletion, transform persistence, and parameterized queries. Overall, the enhancement strengthened my understanding of relational modeling, secure database integration, testing, and deployment.

Verification Summary

Area	Result	Evidence
Relational schema	Passed	Tables, constraints, index, and foreign-key cascade created

CRUD operations	Passed	Create, read, update, list, and delete assertions completed
Security	Passed	Prepared injection-style input remained data and schema stayed intact
Transactions	Passed	Scene replacement executed as one atomic unit
Visual Studio integration	Configured	Database sources included; no additional SQLite .lib required
Windows application run	Final local check	Build, press K/L/P/Delete, and verify console and database file